

MODULE 6

Sortir les secrets du code

Module 6. Sortir les secrets du code

Ce module élimine les secrets codés en dur, sécurise les logs pour ne jamais exposer de données sensibles, et remplace les réponses d'erreur qui fuient des informations techniques.

Objectifs opérationnels

À la fin de ce module, tu auras dans ton application : - ☒ Tous les secrets migrés vers `.env` avec `dotenv` - ☒ `.env` ignoré par Git, `.env.example` versionné - ☒ `JWT_SECRET` régénéré avec `openssl` - ☒ Winston installé, `console.log` remplacés, niveau INFO en prod - ☒ Middleware d'erreur propre, aucun `stack` dans les réponses HTTP - ☒ Section M6 rédigée dans le rapport

Carte du module

ÉTAPE	CONCEPT	ACTION TP	TEMPS
Rappels	Concepts 1-9	Lecture active	15 min
1	Audit secrets	<code>grep</code> + <code>git secrets</code>	10 min
2	Migration <code>.env</code>	<code>dotenv</code> + <code>.gitignore</code>	10 min
3	Vérification historique Git	<code>git log -S</code>	5 min
4	Logger Winston	Remplacer <code>console.log</code>	10 min
5	Error handler	Middleware propre	5 min
6	Test validation	<code>curl</code> + logs	5 min

CONCEPT 1, Pourquoi sortir les secrets du code

Rappel. Git conserve l'intégralité de l'historique de façon immuable. Un secret commité, même supprimé dans un commit suivant, reste lisible dans `git log` ou via GitHub. Les scanners automatiques (truffleHog, Gitleaks, GitGuardian) parcourent en continu les dépôts publics et privés. Les fuites de secrets en production représentent l'un des vecteurs d'intrusion les plus courants. La règle absolue : un secret ne doit jamais

apparaître dans un fichier versionné. Il vit uniquement dans des variables d'environnement ou un gestionnaire de secrets (Vault, AWS Secrets Manager).

Risque concret

`const JWT_SECRET = "secret123"` dans `auth.js` → tout développeur avec accès au dépôt peut forger n'importe quel token JWT. Si le dépôt est public : exposition immédiate à tous.

Exemple de faille

Voir bloc ci-dessous

À vérifier dans l'app

```
grep -rn
"secret\\|password\\|api_key\\|token" vuln-
app/ --include="*.js"
```

Correction

Migrer vers `process.env.JWT_SECRET` chargé depuis `.env`

Test de validation

```
grep -rn "secret123\\|password123" vuln-
app/ ne doit retourner aucun résultat
```

Trace rapport

« L'audit du code source a révélé 3 secrets codés en dur (JWT_SECRET, DB_PASSWORD, SESSION_SECRET). Tous ont été migrés vers des variables d'environnement via dotenv. »

Code vulnérable :

```
// vuln-app/auth.js, AVANT
const JWT_SECRET = "secret123";
const DB_PASSWORD = "admin";
const SESSION_SECRET = "mysecret";

const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '1h' });
```

Code corrigé :

```
// vuln-app/auth.js, APRÈS
require('dotenv').config();
const JWT_SECRET = process.env.JWT_SECRET;

if (!JWT_SECRET) {
  throw new Error('JWT_SECRET manquant dans les variables d\'environnement');
}

const token = jwt.sign({ userId: user.id }, JWT_SECRET, { expiresIn: '1h' });
```

TP, Étape 1 : Audit des secrets dans le code

```
cd vuln-app

# Audit rapide dans le code source
grep -rn "secret\\|password\\|api_key\\|token\\|apikey" . \
  --include="*.js" \
  --exclude-dir=node_modules

# Si git-secrets est installé
git secrets --scan

# Audit avec truffleHog (si disponible)
# pip install truffleHog
# trufflehog git file://. --only-verified
```

✓ **Point de vérification** : noter tous les fichiers et lignes retournés, ce sont les éléments à migrer.

CONCEPT 2, dotenv / Variables d'environnement

Rappel. `dotenv` charge un fichier `.env` dans `process.env` au démarrage de l'application. Le fichier `.env` n'est jamais commité (ajouté à `.gitignore`). Chaque développeur crée son propre `.env` à partir du fichier `.env.example`. En production, les variables sont injectées directement par le système (Docker, Kubernetes, Heroku, PM2 ecosystem). `dotenv` doit être chargé au tout début de l'application, avant tout autre `require`.

Risque concret	Sans dotenv, les secrets se retrouvent dans le code ou dans des variables shell volatiles perdues à chaque redémarrage.
Exemple de faille	<pre>require('dotenv') absent, process.env.JWT_SECRET vaut undefined → JWT signé avec undefined est trivial à forger</pre>
À vérifier dans l'app	Premier <code>require</code> dans <code>vuln-app/app.js</code> , dotenv absent
Correction	<pre>require('dotenv').config() en ligne 1 de app.js</pre>
Test de validation	<pre>node -e "require('dotenv').config(); console.log(process.env.JWT_SECRET ? 'OK' : 'MANQUANT')"</pre>
Trace rapport	« dotenv a été intégré. Les variables d'environnement sont chargées depuis <code>.env</code> au démarrage. Le fichier <code>.env</code> est exclu du versionnement. »

Installation et configuration :

```
npm install dotenv
```

Structure du `.env` :

```
# vuln-app/.env , NE JAMAIS COMMITER CE FICHIER
NODE_ENV=development
PORT=3443

SESSION_SECRET=<générer avec openssl rand -base64 32>
JWT_SECRET=<générer avec openssl rand -base64 64>

DB_PATH=./taskflow.sqlite
LOG_LEVEL=debug
```

Premier require dans app.js :

```
// vuln-app/app.js, ligne 1
require('dotenv').config();

const express = require('express');
// ... reste des imports
```

CONCEPT 3, `.env.example` VS `.env`

Rappel. `.env.example` (ou `.env.template`) est la version publique du fichier de configuration : il liste les clés nécessaires avec des valeurs fictives ou vides. Il est commité dans le dépôt et documente les variables requises pour faire tourner l'application. `.env` contient les vraies valeurs et ne doit jamais être commité. Le `.gitignore` doit explicitement exclure `.env`, `.env.local`, `.env.production`.

Risque concret

Un développeur crée `.env` avec de vraies clés et fait `git add .` sans vérifier → secret exposé à toute l'équipe, potentiellement sur GitHub public.

Exemple de faille

`.gitignore` ne contient pas `.env` → `git status` montre `.env` comme fichier non traqué prêt à être commité

À vérifier dans l'app

`cat vuln-app/.gitignore | grep env` → absent

Correction

Ajouter `.env` au `.gitignore` + créer `.env.example`

Test de validation

`git status` ne doit pas afficher `.env` dans les fichiers non traqués

Trace rapport

« Le fichier `.env.example` a été créé et commité. Le `.gitignore` exclut `.env` et ses variantes. Aucun secret n'apparaît dans le dépôt. »

Mise à jour du `.gitignore` :

```
# Ajouter à vuln-app/.gitignore
.env
.env.local
.env.production
.env.*.local
*.pem          # certificats mkcert (M5)
```

Contenu du `.env.example` :

```
# vuln-app/.env.example, CE FICHIER EST COMMITÉ
NODE_ENV=development
PORT=3443
SESSION_SECRET=REPLACER_PAR_openssl_rand_base64_32
JWT_SECRET=REPLACER_PAR_openssl_rand_base64_64
DB_PATH=./taskflow.sqlite
LOG_LEVEL=info
```

TP, Étape 2 : Migration vers `.env`

```
cd vuln-app
npm install dotenv

# Créer .env avec de vraies valeurs
cp .env.example .env

# Générer des secrets sûrs (voir Concept 4)
openssl rand -base64 32  # pour SESSION_SECRET
openssl rand -base64 64  # pour JWT_SECRET

# Editer .env avec les valeurs générées
# Vérifier que .gitignore exclut .env
git status # .env ne doit PAS apparaître
```

CONCEPT 4, Rotation des secrets

Rappel. Un secret doit être changé régulièrement et immédiatement si une compromission est suspectée. `openssl rand` génère de l'entropie cryptographique solide (contrairement à des chaînes mémorisables). En Node.js, `crypto.randomBytes(64).toString('hex')` produit le même résultat. Changer le `JWT_SECRET` invalide tous les tokens existants, il faut prévoir une période de transition ou une déconnexion forcée des utilisateurs.

Risque concret	<code>JWT_SECRET = "secret123"</code> peut être deviné en quelques secondes par brute-force. Un attaquant qui connaît le secret peut forger des tokens admin.
Exemple de faille	Secret trop court, trop prévisible, identique en dev et en prod
À vérifier dans l'app	Valeur actuelle de <code>JWT_SECRET</code> dans le code
Correction	<code>openssl rand -base64 64</code> → copier dans <code>.env</code>
Test de validation	Tenter de décoder un token existant sur jwt.io avec <code>"secret123"</code> → doit échouer après rotation
Trace rapport	« Le <code>JWT_SECRET</code> a été régénéré avec <code>openssl rand -base64 64</code> (512 bits d'entropie). L'ancien secret était une chaîne de 9 caractères trivialement bruteforçable. »

```
# Générer des secrets sécurisés
echo "SESSION_SECRET=$(openssl rand -base64 32)"
echo "JWT_SECRET=$(openssl rand -base64 64)"

# Alternative Node.js
node -e "const c=require('crypto'); console.log(c.randomBytes(64).toString('hex'))"
```

CONCEPT 5, Logs : ce qu'il ne faut JAMAIS y mettre

Rappel. Les logs sont souvent stockés en clair, transmis vers des agrégateurs externes (Datadog, Splunk, ELK), et accessibles à plusieurs équipes. Ils ne doivent jamais contenir : mots de passe (même hashés), tokens JWT ou sessions, numéros de carte, données de santé, adresses IP complètes en production (RGPD), contenus de champs `password`. La règle : loguer l'événement et l'identifiant, jamais la valeur sensible.

Risque concret	<code>console.log("login:", email, password)</code> → le mot de passe en clair apparaît dans les logs du serveur, accessibles à toute l'équipe ops.
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	<code>grep -rn "console.log.*password\\ console.log.*token\\ console.log.*secret" vuln-app/</code>
Correction	Logger uniquement l'événement : <code>logger.info('login attempt', { email, success: true })</code>
Test de validation	Déclencher un login → vérifier dans les logs qu'aucun mot de passe n'apparaît
Trace rapport	« 4 occurrences de logs exposant des données sensibles (mots de passe, tokens) ont été identifiées et corrigées. Les logs ne contiennent plus que des identifiants et des codes d'événements. »

Code vulnérable :

```
// vuln-app/routes/auth.js, AVANT
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  console.log('login attempt:', email, password);      // ⚠ mot de passe en clair
  console.log('JWT token generated:', token);          // ⚠ token en clair
  console.log('Session:', JSON.stringify(req.session)); // ⚠ session complète
});
```

Code corrigé :

```
// vuln-app/routes/auth.js, APRÈS
app.post('/login', (req, res) => {
  const { email } = req.body;
  // NE PAS logger req.body.password
  logger.info('login_attempt', { email, ip: req.ip });
  // ...
  logger.info('login_success', { userId: user.id, email });
});
```

CONCEPT 6, Logger structuré (Winston)

Rappel. Un logger structuré écrit des lignes JSON avec des champs cohérents (timestamp, level, message, context) plutôt que des chaînes arbitraires. Cela rend les logs filtrables, indexables, et traçables dans des outils comme ELK ou Datadog. Winston est le

logger le plus répandu dans l'écosystème Node.js. Pino est une alternative plus rapide. Les deux produisent des logs JSON par défaut.

Risque concret	<code>console.log</code> ne différencie pas les niveaux, ne peut pas être configuré pour écrire dans un fichier, et n'est pas filtrable en production.
Exemple de faille	Toute l'application utilise <code>console.log</code> , pas de niveau, pas de timestamp standard, impossible à désactiver
À vérifier dans l'app	<pre>grep -rn "console\." vuln-app/ --include="*.js" \ grep -v node_modules</pre>
Correction	Voir configuration Winston ci-dessous
Test de validation	<code>node app.js</code> → logs au format JSON avec timestamp et level
Trace rapport	« <code>console.log</code> a été remplacé par Winston sur toute l'application. Les logs sont désormais structurés (JSON), horodatés, et configurables par niveau. »

Installation et configuration Winston :

```
npm install winston
```

```
// vuln-app/logger.js, NOUVEAU FICHIER
const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: false }), // stack uniquement en debug
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error',
    }),
    new winston.transports.File({
      filename: 'logs/app.log',
    }),
  ],
});

module.exports = logger;
```

```
// Dans chaque fichier de route, remplacer console.log
const logger = require('../logger');

// AVANT : console.log('user created', user)
// APRÈS :
logger.info('user_created', { userId: user.id, email: user.email });
logger.warn('login_failed', { email, reason: 'invalid_password', ip: req.ip });
logger.error('db_error', { operation: 'INSERT', table: 'tasks' });
// ⚠️ Jamais : logger.info('login', { password: req.body.password })
```

CONCEPT 7, Niveaux de logs

Rappel. Les niveaux de log permettent de filtrer le bruit selon l'environnement. En développement, on active `DEBUG` pour voir tous les détails. En production, on passe à `INFO` ou `WARN` pour ne conserver que les événements significatifs. Winston utilise par défaut : `error` (0) > `warn` (1) > `info` (2) > `http` (3) > `verbose` (4) > `debug` (5) > `silly` (6). Un niveau configuré à `info` n'écrira pas les messages `debug`.

Risque concret	<code>LOG_LEVEL=debug</code> en production → les logs exposent des détails internes (requêtes SQL, contenu des objets) qui facilitent la reconnaissance par un attaquant ayant accès aux logs.
Exemple de faille	<code>LOG_LEVEL</code> non défini → Winston utilise <code>silly</code> par défaut → tout est logué
À vérifier dans l'app	Variable <code>LOG_LEVEL</code> absente de <code>.env</code>
Correction	<code>LOG_LEVEL=info</code> dans <code>.env.production</code> , <code>LOG_LEVEL=debug</code> dans <code>.env.development</code>
Test de validation	Avec <code>LOG_LEVEL=info</code> , les <code>logger.debug(...)</code> n'apparaissent pas dans la sortie
Trace rapport	« Le niveau de log est contrôlé par la variable <code>LOG_LEVEL</code> . En production, seuls les niveaux <code>info</code> et supérieurs sont enregistrés. »

```
// Configuration dans logger.js, déjà gérée par :
level: process.env.LOG_LEVEL || 'info',
// 'info' est la valeur par défaut sécurisée
```

CONCEPT 8, Erreurs côté API : ne pas exposer le stack

Rappel. En développement, afficher le `stack trace` dans la réponse HTTP est pratique pour déboguer. En production, c'est une fuite critique : le stack révèle les noms de fichiers internes, les versions de bibliothèques, et la structure de l'application, autant d'informations utiles pour un attaquant. La règle : l'utilisateur reçoit un message générique, le stack est écrit dans les logs internes. Un middleware d'erreur centralisé dans Express gère cette séparation.

Risque concret	<pre>res.status(500).json({ stack: err.stack })</pre> → l'attaquant voit le chemin complet <pre>/home/ubuntu/app/node_modules/sequelize/ ...</pre> et identifie les versions vulnérables
Exemple de faille	Voir bloc ci-dessous
À vérifier dans l'app	<pre>grep -rn "err.stack\ error.stack" vuln-app/</pre>
Correction	Middleware d'erreur centralisé, stack logué, jamais envoyé au client
Test de validation	<pre>curl -X POST https://localhost:3443/error-test</pre> → réponse JSON sans <code>stack</code>
Trace rapport	« Le middleware d'erreur a été refactorisé. Les stack traces ne sont plus incluses dans les réponses HTTP. Les erreurs sont loguées avec Winston pour analyse interne. »

Code vulnérable :

```
// vuln-app/app.js, AVANT
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({
    error: err.message,
    stack: err.stack,      // ⚠ fuite totale de l'arborescence
    query: err.sql,        // ⚠ fuite de la requête SQL
  });
});
```

Code corrigé :

```
// vuln-app/app.js, APRÈS
const logger = require('./logger');

app.use((err, req, res, next) => {
  // Log complet en interne (jamais envoyé au client)
  logger.error('unhandled_error', {
    message: err.message,
    stack: err.stack,
    url: req.url,
    method: req.method,
    userId: req.session?.userId,
  });

  // Réponse générique au client
  const statusCode = err.statusCode || 500;
  const isProd = process.env.NODE_ENV === 'production';

  res.status(statusCode).json({
    error: isProd ? 'Une erreur interne s\'est produite.' : err.message,
    // stack JAMAIS inclus
  });
});
```

CONCEPT 9, Atelier : Sécurisation du stockage des identifiants (bcrypt + dotenv)

Rappel. Les mots de passe ne doivent jamais être stockés en clair ni simplement hashés (MD5, SHA1). `bcrypt` est un algorithme de hachage adaptatif qui intègre un sel aléatoire et un facteur de coût. Plus le facteur est élevé, plus le calcul est lent, ce qui résiste au brute-force. La valeur recommandée pour 2024 est entre 12 et 14. Combiné à `dotenv` pour les secrets de l'application, `bcrypt` assure que même une fuite de la base de données ne compromet pas les mots de passe des utilisateurs.

Risque concret	Mots de passe stockés en clair ou avec MD5 → dump de la BDD = liste de tous les mots de passe exploitables immédiatement
Exemple de faille	<pre>db.run("INSERT INTO users VALUES (?, ?)", [email, password])</pre> , mot de passe en clair
À vérifier dans l'app	<pre>grep -n "password" vuln- app/routes/auth.js</pre> , chercher INSERT sans bcrypt
Correction	Voir bloc ci-dessous
Test de validation	<pre>sqlite3 vuln-app/taskflow.sqlite "SELECT password FROM users LIMIT 1"</pre> → doit retourner <code>\$2b\$12\$...</code>
Trace rapport	« Les mots de passe sont désormais hashés avec bcrypt (facteur 12) avant stockage. Aucun mot de passe en clair ne figure dans la base de données. »

Code sécurisé :

```
// vuln-app/routes/auth.js, APRÈS
const bcrypt = require('bcrypt');
require('dotenv').config();

const BCRYPT_ROUNDS = parseInt(process.env.BCRYPT_ROUNDS) || 12;

// Inscription
app.post('/register', async (req, res) => {
  const { email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, BCRYPT_ROUNDS);
  db.run(
    'INSERT INTO users (email, password) VALUES (?, ?)',
    [email, hashedPassword],
    (err) => {
      if (err) return res.status(400).json({ error: 'Email déjà utilisé' });
      logger.info('user_registered', { email });
      res.status(201).json({ message: 'Compte créé' });
    }
  );
});

// Connexion
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  logger.info('login_attempt', { email, ip: req.ip }); // pas de password dans le
  log
  db.get('SELECT * FROM users WHERE email = ?', [email], async (err, user) => {
    if (!user || !(await bcrypt.compare(password, user.password))) {
      logger.warn('login_failed', { email, ip: req.ip });
      return res.status(401).json({ error: 'Identifiants invalides' });
    }
    req.session.userId = user.id;
    logger.info('login_success', { userId: user.id, email });
    res.json({ message: 'Connecté' });
  });
});
```


TP Complet M6 (45 min)

Étape 1, Audit secrets dans le code (10 min)

```
cd vuln-app

# Audit du code source
grep -rn "secret\\|password\\|api_key\\|token\\|apikey" . \
  --include="*.js" \
  --exclude-dir=node_modules \
  --color=always

# Lister les résultats dans un fichier pour le rapport
grep -rn "secret\\|password" . \
  --include="*.js" \
  --exclude-dir=node_modules \
  > ../proofs/M6/audit-secrets.txt
```

✓ **Point de vérification** : au moins 3 secrets identifiés dans le code source.

Étape 2, Migrer les secrets vers .env (10 min)

```
npm install dotenv bcrypt winston

# Créer le .env à partir de l'exemple
cp .env.example .env

# Générer des secrets sécurisés
echo "SESSION_SECRET=$(openssl rand -base64 32)"
echo "JWT_SECRET=$(openssl rand -base64 64)"

# Copier les valeurs dans .env
# Ajouter .env au .gitignore
echo ".env" >> .gitignore
echo ".env.*.local" >> .gitignore

git status # .env ne doit pas apparaître
```

Étape 3, Vérifier l'historique Git (5 min)

```
# Rechercher si un secret a déjà été commité
git log -S "secret123" --all --oneline
git log -S "password123" --all --oneline
git log --all --full-history -- "**/.env"

# Si un secret a été commité : documenter dans le rapport
# (remediation = rotation immédiate du secret + BFG Repo Cleaner si dépôt public)
```

⚠ **Si git log -S retourne des résultats** : noter dans le rapport que le secret a été exposé dans l'historique et qu'une rotation a été effectuée.

Étape 4, Installer Winston et remplacer console.log (10 min)

```
# Créer le fichier logger
# (copier le contenu du Concept 6)

# Audit des console.log à remplacer
grep -rn "console\." . \
  --include="*.js" \
  --exclude-dir=node_modules

# Remplacer dans les fichiers routes/auth.js, routes/tasks.js, app.js
# Utiliser le pattern : const logger = require('../logger');
```

Étape 5, Refactor du middleware d'erreur (5 min)

Remplacer le middleware d'erreur existant dans `app.js` par le code corrigé du Concept 8.

Étape 6, Test de validation (5 min)

```
# Provoquer une erreur intentionnelle
curl -k -X GET https://localhost:3443/route-inexistante
# Réponse attendue : {"error":"Une erreur interne s'est produite."}
# Pas de stack dans la réponse

# Vérifier les logs
tail -f logs/app.log
# Le stack doit apparaître dans les logs, pas dans la réponse HTTP

# Vérifier la base de données
sqlite3 taskflow.sqlite "SELECT email, password FROM users LIMIT 3;"
# Les mots de passe doivent commencer par $2b$12$
```

✓ **Points de vérification finaux :** - `grep -rn "secret123\\|password123" . --include="*.js"`
→ aucun résultat - `cat logs/app.log` → logs JSON avec timestamp et level - Réponse d'erreur HTTP → message générique, pas de stack - BDD → mots de passe hashés bcrypt

Mini-quiz M6 (6 QCM)

Q1. Pourquoi supprimer un secret d'un commit ne suffit pas ? - A. Car Git compresse les fichiers - B. Car l'historique Git est immuable, le secret reste visible dans les commits précédents - C. Car GitHub réindexe automatiquement - D. Car les branches gardent une copie

Q2. Quel est le facteur de coût bcrypt recommandé en 2024 ? - A. 4-6 (performance maximale) - B. 8-10 (équilibre) - C. 12-14 (sécurité recommandée) - D. 20+ (sécurité maximale)

Q3. Que doit contenir `.env.example` ? - A. Les vraies valeurs de production - B. Les clés avec des valeurs fictives ou vides, jamais les vrais secrets - C. Uniquement les clés sans valeur - D. Une copie chiffrée des secrets

Q4. Quel niveau de log est recommandé en production ? - A. debug (tout voir) - B. silly (maximum de détails) - C. info (événements significatifs uniquement) - D. error (seulement les erreurs critiques)

Q5. Que doit retourner un endpoint en erreur 500 en production ? - A. Le message d'erreur complet avec stack - B. Le nom du fichier et la ligne d'erreur - C. Un message générique, le détail va dans les logs internes - D. Le contenu de la requête SQL ayant échoué

Q6. `console.log('login:', email, password)` est problématique car : - A. `console.log` est trop lent en production - B. Le mot de passe apparaît en clair dans les logs, accessibles à l'équipe ops - C. Le format n'est pas JSON - D. Email et password ne peuvent pas être concaténés

Corrigé

Q	RÉPONSE	POURQUOI
Q1	B	<code>git log -S</code> retrouve n'importe quelle chaîne dans l'historique
Q2	C	12-14 : assez lent pour résister au GPU brute-force, assez rapide pour l'UX
Q3	B	<code>.env.example</code> est un modèle sans secrets réels, il documente sans exposer
Q4	C	<code>info</code> capture les événements métier sans les détails de debug
Q5	C	Séparer ce que l'utilisateur voit de ce que les ops voient
Q6	B	Les logs sont des fichiers, potentiellement exfiltrables, jamais de PII/secrets

Livrable du module

- ☐ `npm install dotenv bcrypt winston` effectué
- ☐ `.env` créé avec secrets générés via `openssl rand`
- ☐ `.env` ajouté au `.gitignore`, `.env.example` commité
- ☐ `grep -rn "secret123" vuln-app/` → aucun résultat
- ☐ `logger.js` créé, `console.log` remplacés dans toutes les routes
- ☐ Middleware d'erreur centralisé, pas de stack dans les réponses
- ☐ BDD : mots de passe préfixés `$2b$12$`
- ☐ Captures dans `proofs/M6/`
- ☐ Tag Git : `git tag M6-secrets-logs && git push --tags`
- ☐ Section M6 rédigée dans le rapport

Erreurs fréquentes à éviter

- Charger `dotenv` après d'autres modules qui utilisent `process.env` → les variables ne sont pas encore définies. Toujours mettre `require('dotenv').config()` en ligne 1.

- Commiter `.env` par erreur avec `git add .` → utiliser `git add -p` pour une revue sélective.
- Logger `req.body` entier → peut contenir les mots de passe. Lister explicitement les champs à logger.
- Utiliser `LOG_LEVEL=debug` en production → exposition des détails internes.
- Oublier de créer le dossier `logs/` → Winston échoue silencieusement sur certaines configurations.

Ressources

- OWASP Top 10, A02:2021 Cryptographic Failures : https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- OWASP Top 10, A09:2021 Security Logging and Monitoring Failures : https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
- OWASP Logging Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- Winston documentation : <https://github.com/winstonjs/winston>
- dotenv documentation : <https://github.com/motdotla/dotenv>
- bcrypt npm : <https://www.npmjs.com/package/bcrypt>
- truffleHog (scan secrets Git) : <https://github.com/trufflesecurity/trufflehog>